Understanding Joins *in* Hadoop

Those who have just begun the study of Hadoop might have come across different types of joins. This article briefly discusses normal joins, map side joins and reduce side joins. The differences between map side joins and reduce side joins, as well as their pros and cons, are also discussed.

Normally, the term join is used to refer to the combination of the record-sets of two tables. Thus when we run a query, tables are joined and we get the data from two tables in the joined format, as is the case in SQL joins. Joins find maximum usage in Hadoop processing. They should be used when large data sets are encountered and there is no urgency to generate the outcome. In case of Hadoop common joins, Hadoop distributes all the rows on all the nodes based on the join key. Once this is achieved, all the keys that have the same values end up on the same node and then, finally, the join at the reducer happens. This scenario is perfect when both the tables are huge, but when one table is small and the other is quite big, common joins become inefficient and take more time to distribute the row.

While processing data using Hadoop, we generally do it over the map phase and the reduce phase. Thus there are mappers and reducers that do the job for the map phase and the reduce phase. We use map reduce joins when we encounter a large data set that is too big to use data-sharing techniques.

Map side joins

Map side join is the term used when the record sets of two tables are joined within the mapper. In this case, the reduce phase is not involved. In the map side join, the record sets of the tables are loaded into memory, ensuring a faster join operation. Map side join is convenient for small tables and not recommended for large tables. In situations where you have queries running too frequently with small table joins you could experience a very significant reduction in query computation time.

Reduce side joins

Reduce side joins happen at the reduce side of Hadoop processing. They are also known as repartitioned sort merge joins, or simply, repartitioned joins or distributed joins or common joins. They are the most widely used joins. Reduce side joins happen when both the tables are so big that they cannot fit into the memory. The process flow of reduce side joins is as follows:

1. The input data is read by the mapper, which needs to be combined on the basis of the join key or common column.
2. Once the input data is processed by the mapper, it adds a tag to the processed input data in order to distinguish the input origin sources.
3. The mapper returns the intermediate key-value pair, where the key is also the join key.
4. For the reducer, a key and a list of values is generated once the sorting and shuffling phase is complete.
5. The reducer joins the values that are present in the generated list along with the key to produce the final outcome.

The join at the reduce side combines the output of two mappers based on a common key. This scenario is quite synonymous with SQL joins, where the data sets of two tables are joined based on a primary key. In this case we have to decide which field is the primary key.

There are a few terms associated with reduce side joins:

1. *Data source:* This is nothing but the input files.
2. *Tag:* This is basically used to distinguish each input data on the basis of its origin.
3. *Group key:* This refers to the common column that is used as a join key to combine the output of two mappers.